KNUST | Department of Computer Engineering

COE 454 Software Engineering II Exam Study Guide

Project: Regimens | Client: Outstretched Arm Enterprise
Team of 7 | Lecturer: Eliel Keelson
Based on Lectures 1, 2 (architecture), 5 (business), 6 (licensing)
plus the live system: React + Express + Postgres, Vercel + Render.

How to use this on Friday

1. Learn the DEFINITION from the slides first. Then add one Regimens sentence. Examiners mark both.
2. Never say 'we used React' as if that were architecture. Architecture is how parts communicate and who owns data.
3. If you do not know, say the principle, then map it honestly. Do not invent a microservice you did not build.
4. Oral exams reward calm structure: definition, why it mattered, what we did, what we would do next.
5. Red boxes below are likely questions. Green-tinted answers are model answers you should shorten in speech.

0. Thirty-second project pitch (memorise)

Regimens is an offline-first inventory and sales system for Outstretched Arm Enterprise, a stationery shop that tracked stock in a notebook. We replace that with login, products, categories, sales, reports, profit, backup, and owner settings. The frontend is React on Vercel. The API is Express on Render with PostgreSQL. Cost is stored in FIFO stock layers so leftover old stock keeps its arrival cost while sales use today's shelf price. Seven joint authors under Ghana Copyright Act 690.

Team: Hafsa (docs / backend support), Albert (backend), Francis and Raymond (database), David (UI/UX), Peter and Benjamin (frontend).

1. Lecture 1 - What is SE, SDLC, requirements

Definitions they can ask word-for-word

Software Engineering is a disciplined, systematic approach to the design, development, maintenance, and testing of software, aiming at high-quality systems that meet user requirements and are reliable, maintainable, and scalable.

SDLC is the structured process used to design, develop, and test software so that it meets customer expectations, finishes within time and cost, and works in the planned IT environment.

Traditional SDLC steps: Problem definition; Requirements gathering and analysis; Planning and designing; Development/implementation; Testing; Deployment; Maintenance and support.

How we actually walked SDLC (honest)

- Problem: notebook tracking, lost stock, untrusted profit when prices change, no work offline.
- Requirements: interviews/observation of a shop, user stories, MoSCoW after client demo.
- Design: ERD, API schema, UI/UX guide, ADRs for React, Express, Postgres, JWT, offline queue.
- Implementation: monorepo folders backend/, frontend/, database/ with domain routes.
- Testing: API smoke scripts, demo walkthrough, client demo (not a full automated test suite - say that).
- Deployment: Vercel SPA + Render API/Postgres. Maintenance: migrations, password change, backups.

Methodologies vs SDLC

SDLC is the STAGES of work. A methodology is HOW you move through those stages. Waterfall goes through them once, in order. Agile/Scrum repeats short cycles of the same stages. Prototyping emphasises early UI. Spiral emphasises risk. We were closest to Agile with a modular-monolith MVP: working software every week, main kept green, branches per task, client feedback then MoSCoW. We were NOT a pure Waterfall shop.

Functional vs non-functional

Functional = what the system does (features, behaviour, user interactions). Non-functional = quality attributes (performance, security, usability, reliability, maintainability, scalability).

Banking example from slides maps 1:1: registration/auth, account management, transfers, alerts are functional; HTTPS, response time, availability, usability, compliance are non-functional. For us, 'record a sale' is functional; 'sale still works after the network drops' is non-functional (reliability + offline).

Q: Give three functional and three non-functional requirements of Regimens.

A: Functional: (1) Owner logs in with username/password and receives a JWT. (2) Record a sale that decrements stock and stamps unit_price and unit_cost on each line. (3) Show gross profit by day/week/month/year. Non-functional: (1) Security - hashed passwords, JWT, rate limits, CORS, parameterized

SQL. (2) Reliability/offline - Dexie queue + /api/sync, last-write-wins. (3) Usability - Ghana cedis, AppShell, low-stock colours, three-step sale. Extra: Render cold start is a known availability NFR we only partly solved with keep-alive pings.

Q: How did you elicit requirements?

A: Stakeholder contact with the shop (problem domain), document analysis of the notebook process, use-case / user-story style ('as owner I record a sale'), prototyping the React screens, and a client demo plus feedback form. Non-functionals came from observing the counter (speed, offline, simple UI) and from course constraints (JWT, backups, no secrets in git). We then classified requested changes with MoSCoW.

2. Lecture 2 - Architecture (this will be asked)

Architecture is the set of significant design decisions about how the system is organised: how parts communicate, who owns data, and how it grows. It is NOT 'we used JavaScript'. Changing a button colour is cheap. Moving a load-bearing wall (for example splitting the database) is not.

Monolith vs microservices vs modular monolith

- Monolith: one deployable app; UI, logic, data often together; one process, usually one DB. Simple for MVP, small team. Risk: one bug can take everything down; you scale the whole app.
- Microservices: one service per business capability, independently deployed, talk over HTTP/queues. Hard: many deploys, network failures, distributed data. Rule of thumb from class: do NOT start here.
- Modular monolith (recommended): ONE deployable unit, internally organised by DOMAIN (auth, products, sales) not only by technical layer (all controllers in one folder). Later you can lift one folder into a service.

Q: Is Regimens a monolith or microservices? Three-word justification.

A: Modular monolith (plus a separate SPA). Small team, well-defined shop problem, no need to scale sales independently yet. One Express process owns auth, products, categories, sales, reports, backup, sync. One Postgres. The React app is a client, not a second microservice. We split HOSTING (Vercel vs Render) for cost and sleep, not because we extracted domains. If lunch-rush sales ever needed isolation, we would extract /sales, not rewrite.

Q: What domains would you put under src?

A: On the API: /auth, /products, /categories, /sales, /reports, /backup, /sync. Frontend pages mirror those domains. That matches the chop-bar exercise: /auth, /menu, /orders, /notifications, /reports.

Q: If 500 people used it tonight, what fails first?

A: The free Render API (one instance, cold start, rate limits) and a single Postgres. Not React static files on Vercel. Scale sequence from class: (1) optimise queries/indexes, (2) bigger server, (3) more servers if we stay stateless JWT, (4) extract the hottest module. We would NOT jump to microservices because the architecture looks inelegant.

Tech stack (say why, not just names)

- React + Vite: fast UI iteration, Vercel-native, team skill.
- Express + JWT: simple REST contract matching the API schema; mock vs Postgres behind one interface.
- PostgreSQL: money as NUMERIC, migrations, FK integrity for sales lines.
- Dexie: client-side queue when navigator.onLine is false.

USE_MOCK_DB=true was an architecture decision: frontend was not blocked on Francis/Raymond. Same routes, two data layers.

3. Quality, testing, VCS, maintenance, ethics

Lecture 1 listed these as areas of concern. Expect a short question even if there was no dedicated PDF in the pack you sent.

- Testing: functional tests check features (login, sale stamps cost). NFR tests check qualities (rate limit, CORS, offline queue). We used demo scripts and a client walkthrough more than unit tests. That is a gap; say we would add API tests next.
- Version control: GitHub, main always working, branch area/short-description, PR + one reviewer. Configuration: env vars, never commit .env.
- Maintenance: numbered SQL migrations (008 added stock layers), seed, backup/restore. Adaptive maintenance = FIFO layers after price-change risk.
- Ethics: hashed passwords, no username enumeration, owner-only backup, change demo password, honest profit (do not rewrite old stock cost).

4. Lecture 5 - Business of SE (high exam weight)

IP and Ghana Copyright Act 690

Software is a literary work. Copyright exists at creation; no registration required. Default owner is the author. Payment does NOT transfer copyright unless the contract has an explicit IP assignment. Seven teammates = joint authors. Economic rights (copy, sell, license, derivatives) CAN be assigned in writing. Moral rights (be named as author, object to derogatory treatment) cannot be taken away.

Q: If we commercialise Regimens, who owns it?

A: All seven of us, jointly, until a written assignment. Proportions are not automatic 1/7 unless we agree; default is joint ownership. The team charter should say this. Client Outstretched Arm does NOT own the code just because they were the problem domain, unless they paid under a contract with IP assignment. Best practice: client owns after final payment; we keep the right to reuse non-client-specific pieces (auth boilerplate, FIFO helper).

Pricing models

- One-time license: pay once, use the tool. Fits a bespoke shop system. Example from class: GHS 8,000 inventory.
- SaaS: GHS 250/month hosted. Fits if we multi-tenant many shops.
- Freemium: free up to N products; pay for profit analytics.
- Usage-based: per sale recorded. Aligns with value; harder to explain to a stationery shop.

For THIS client, honest answer: one-time license plus a small hosting retainer, OR a cheap SaaS if we host on Render. Do not quote GHS 1,500 as a student 'feeling'. Use five steps: cost floor, market rate, value (time saved vs notebook), positioning, ask what they expected to pay. Include discovery, revisions, QA (+20-30%), deploy, docs, 30-90 day support, 15-20% uncertainty.

Contracts

Must contain: scope (point at requirements), payment schedule, IP assignment, change process, support period, termination, limitation of liability (hosting outages, ECG, client misuse). Electronic Transactions Act 2008: email agreement can bind. Fixed-price: client certainty, WE hold overrun risk, needs frozen scope. T&M: they pay hours, they hold overrun risk. Ghana practice: 30-50% deposit, milestone at MVP, final on acceptance. Unpaid: reminder, demand letter, ADR Act 2010, Small Claims under GHS 10,000.

Defect vs change (memorise the test)

Defect: it does not do what the signed requirements said (login fails on valid credentials; profit uses today's cost for old sales). Free in support window. Change: not in scope ('add payroll', 'new brand colours'). Price it, written approval, then build. Never verbal yes. Process: assess, document, price, written approval, then build. Scope freeze: new ideas go to backlog / Won't Have this phase, not into the current sprint.

Q: Client tomorrow asks for a payroll module. Word-for-word response?

A: 'That is a good idea. It is not in the agreed requirements, so it is a change, not a defect. Let me assess impact on database, API, and UI, write a change request with cost and time, and send it for your written approval. We will not start until you confirm. Meanwhile we finish the agreed inventory and sales MVP. Payroll can be first in phase two.'

MoSCoW

- M Must: unusable or unsafe without it. Example: FIFO cost layers if they change supplier prices.
- S Should: important but shop can run. Example: staff role with less access than owner; receipts.
- C Could: nice if time. Example: stronger low-stock notifications.
- W Won't this phase: native Android app. Promise to consider later, not a permanent insult.

Clients and complaints

First impression is a deliverable: punctual, clear requirements doc. Weekly two-line update. WhatsApp for short; email for record. Complaint: (1) acknowledge, (2) own what is yours, (3) explain what is not (Render sleep), (4) concrete next step and time. Never go silent. Handover = start of retainer: hosting + support monthly, phase-two Could/Won't list, ask for referrals.

Career + scaling

Great developer: knows who owns the work, prices value, writes agreements, communicates first, handles complaints, turns delivery into the next conversation. Scale product when REAL load hurts (slow queries, errors at rush), not because a blog likes microservices. Optimise, then vertical, then horizontal, then extract. Scale business when demand exceeds quality capacity; too early = hire with no revenue; too late = missed deadlines and declining quality.

5. Lecture 6 - Open-source licensing

Copyright defaults to all rights reserved, so collaboration needs a licence that grants permission in advance. Permissive (MIT, ISC, BSD, Apache 2.0): use commercially, keep notice. Apache adds patent grant + modification notice. Weak copyleft: MPL = file level; LGPL = link OK, modifying the library means share those changes. Strong copyleft: GPL = distribute the product, whole product must be GPL. AGPL = even SaaS/network use can force source sharing. CC is for docs/images, NOT software. CC BY-NC cannot go in a commercial product. Dual licence (MySQL, Qt): GPL free or pay for proprietary. SSPL/BUSL are source-available, not OSI open source.

History hooks: Stallman GNU 1983; GPL v1 1989; Linux GPL v2 1991; 'open source' + OSI 1998; Apache 2.0 2004; GPLv3/AGPLv3 2007; MongoDB SSPL 2018.

Q: Could any library stop Outstretched Arm selling Regimens?

A: We should run npx license-checker --summary before handover. Our main stack is permissive: Express, React, Vite, axios, jsonwebtoken, pg, helmet are MIT-class; lucide-react is ISC (treat like MIT); Dexie is Apache 2.0 (still commercial-safe). Recharts is MIT. That is the SAFE side of the spectrum. If we had pulled a GPL chart library, delivering a closed shop system would infect the product: client could demand source. Fix: replace with MIT, record in an ADR. Icons: no CC BY-NC. Attach the audit to acceptance.

Q: A dependency switches MIT to AGPL mid-project. Options?

A: (1) Pin the last MIT version and stop upgrading. (2) Replace the library. (3) Accept AGPL and open the SaaS source - usually unacceptable for a closed shop tool. We would pin or replace and log it in the ADR.

6. Drill pack - more likely orals

Q: Why JWT and not sessions in a cookie only?

A: SPA on Vercel, API on Render: stateless JWT lets any API instance validate the token without sticky sessions. We still use HTTPS in production, short expiry (12h), strong JWT_SECRET. Refresh-token rotation was listed as future hardening.

Q: Why stamp unit_cost on sale_items AND keep stock layers?

A: Stamping protects completed sales when product cost is edited later. Layers protect UNSOLD quantity: restock at a new cost opens a new lot; FIFO consumes oldest first; leftover old books keep 2.50 while new ones cost 4.00; shelf price can rise independently. Profit = qty x (stamped sell - stamped cost).

Q: Waterfall or Agile, and a risk of your choice.

A: Agile-leaning MVP. Risk we accepted: incomplete automated tests. Risk we avoided: starting with microservices. Risk we mitigated: mock data layer so the team was not blocked.

Q: Usability vs security trade-off.

A: Demo login owner/owner1234 is usable for class and dangerous in production. NFR security says change it in Settings. Rate limiting slows attackers and can lock out a shop if too aggressive. We rate-limit login more than ordinary GETs.

Q: What is in a good ADR?

A: Context, decision, alternatives, consequences. Example: 'Use modular Express + Postgres not Firebase, because we need SQL NUMERIC money, migrations, and an ERD the database teammates own.'

Q: Maintainability: give evidence.

A: Numbered migrations, mock/real implementing one interface, domain routes, README per folder, env examples, demo script. Weak evidence: some Word docs not in the clone; we should keep ERD in-repo.

7. One-page cheat - say this if your mind goes blank

SE = systematic build + maintain + test for quality.

SDLC = problem, requirements, design, build, test, deploy, maintain.

Functional = what it does. Non-functional = how well (secure, fast, usable, offline).

Architecture = load-bearing decisions. We built a modular monolith API + SPA, not microservices.

Act 690 = we jointly own until a written IP clause. Moral rights stay.

Defect = broken requirement, free fix. Change = new scope, price + written yes.

MoSCoW = Must / Should / Could / Won't this phase.

MIT/Apache/ISC = commercial OK. GPL/AGPL = can force source. Audit before ship.

Price = cost floor + market + value + 15-20% buffer. Deposit before serious work.

Complaint = hear, own, explain hosting, next step with a time. Never disappear.

Regimens = notebook to trusted profit + offline sales for a Ghana stationery shop.

8. Night-before checklist

- Rehearse the 30-second pitch until it is boring.
- Draw the architecture: Browser -> Vercel React -> Render Express -> Postgres. Offline box: Dexie -> /sync.
- Name seven teammates and roles without looking.
- List 5 functional, 5 NFR, 4 SDLC artefacts (ERD, API schema, ADR, UI guide).
- Explain FIFO with numbers: 50 books at 2.50, restock 5 at 4.00, sell 50 then 5.
- Recite defect vs change and the five change-request steps.
- Recite MIT vs GPL vs AGPL in one sentence each.
- If asked something we did not do (full TDD, microservices, formal contract), say the principle and the gap. Honesty scores.